

PREFLOW: Safe Preemptive Scheduling for LLM prefill

LLM serving systems process prefill requests with heterogeneous prompt lengths and bursty arrivals. Under FCFS, a long prompt can block many shorter prompts, causing head-of-line (HOL) blocking. Preemption can reduce HOL blocking, but it redistributes queueing delay across requests. Unconstrained reordering can substantially improve mean latency while imposing severe delay on individual long requests.

Our synthetic long-context stress test demonstrates this tradeoff. SRPT improves mean time to first token (TTFT) by 84.6% over FCFS, but the worst matched request takes $7.52\times$ as long as it does under FCFS. PREFLOW with 120% slack retains a 78.2% mean improvement and limits the observed worst matched slowdown to $2.37\times$. With 30% slack, it still improves mean TTFT by 68.2%, while the observed worst slowdown is $1.37\times$.

The goal is to retain most of the latency benefit of preemption while explicitly limiting the additional delay imposed on individual requests. PREFLOW separates these concerns. A stretch-oriented, work-aware efficiency objective decides which request should run next. A separate feasibility test determines whether that action remains safe for every request already in the system. High-value work can overtake only while the queue has enough slack to absorb the delay.

The synthetic stress test

The workload is intentionally severe and designed to make queue ordering visible. It mixes prompt lengths that differ by $25\times$ across bursty arrival phases. This is not a persistent-overload experiment: queues build during bursts and later drain.

- The trace contains 1,000 requests generated by a Markov-modulated Poisson process, averaging 0.5 requests/s over a 2,000-second arrival span.
- Prompt buckets are 4K, 8K, 16K, 32K, 65K, and 100K tokens, with shares of 30%, 25%, 20%, 14%, 8%, and 3%.
- Each request asks for 128 output tokens.
- One TP=2 prefill instance and one TP=2 decode instance run across four Ascend NPUs. Prefill uses fixed 4,096-token chunks, eight resident requests, and one scheduled request per step.
- The model is Qwen3-30B-A3B-Instruct-2507 with dummy weight loading. Prefix caching is disabled.

The six prompt classes span $25\times$ in token count, but FCFS compresses their mean TTFT into a narrow 116.6–123.8 second range. Their execution costs still differ substantially; during bursts, queueing dominates total TTFT strongly enough to hide much of the underlying cost difference.

PREFLOW-50 lets short requests pass while earlier deadlines still have enough

slack. Mean TTFT falls to 5.0 seconds for 4K prompts and then grows with prompt size, reaching 186.9 seconds at 100K. PREFLOW does not remove congestion; it redistributes waiting time. Short prompts benefit dramatically, while the largest prompts absorb more delay. This is the benefit/harm tradeoff examined later in the post. The grouped bars use observed wall-clock TTFT from the same matched benchmark requests.

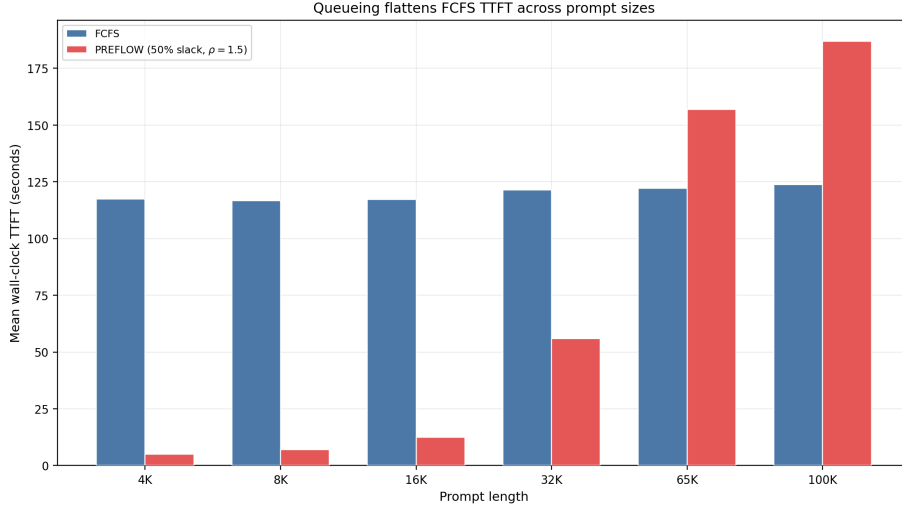


Figure 1: FCFS and PREFLOW-50 mean wall-clock TTFT by prompt length

All 11 policy configurations completed the same 1,000 requests with identical request IDs, prompt lengths, and scheduled arrival times. The experiment is intentionally controlled and is designed to isolate the scheduling tradeoff.

Why aging does not provide a completion contract

Aging is a common way to soften starvation in work-aware scheduling. PrefillOnly is the representative aging baseline in this experiment: predicted work favors short requests, while a linear waiting-time term gradually raises the priority of requests left behind.

The coefficient λ controls how quickly waiting overcomes the work ranking. A small value stays close to shortest-work scheduling. A large value pushes the policy back toward FCFS. This creates an empirical efficiency/protection tradeoff, but λ does not specify a maximum per-request slowdown. The effective protection also changes with the arrival process, prompt distribution, chunk size, and hardware.

PREFLOW gives the operator a parameter with direct semantics instead. The parameter controls the maximum modeled inflation over the FCFS service a request could expect from the state it saw at arrival.

PREFLOW in three steps

PREFLOW makes one scheduling decision at each chunk boundary.

1. Freeze the request’s arrival-state FCFS baseline. For request i ,

B_i = predicted FCFS completion latency from the state at arrival.

This baseline includes unfinished work ahead of the request and counts any outstanding chunk exactly once. Later arrivals cannot enlarge it.

2. Convert that baseline into an immutable completion entitlement:

$$D_i = a_i + \rho B_i.$$

Here a_i is the logical arrival time. Setting $\rho = 1.3$ allows at most 30% modeled completion inflation over that request’s arrival-state FCFS baseline. The experiments use $\rho \in \{1.3, 1.5, 1.8, 2.2\}$.

3. Rank requests by the work-aware objective and execute the highest-ranked chunk that preserves every fixed deadline.

This is the central difference from aging. PREFLOW defines which schedules are legal, then optimizes aggressively inside that feasible region. Aging changes the priority score without imposing the same completion constraint.

The implementation ranks safe requests by

$$\pi_i(t) = \frac{1}{p_i r_i(t)},$$

where p_i is the request’s isolated prefill work and $r_i(t)$ is its remaining work. This is the weighted-shortest-remaining-work index for total stretch. Stretch is useful when request sizes differ by orders of magnitude: an extra second is an $11\times$ completion time for a request that needs 0.1 seconds in isolation, but only $1.1\times$ for one that needs 10 seconds.

The score $1/(p_i r_i)$ is the weighted-shortest-remaining-work priority induced by the stretch objective. PREFLOW applies that priority only inside the feasible action set. If the highest-ranked chunk is unsafe, it tries the next candidate. An earliest-deadline action is the rescue choice when the queue has no slack for a preferred overtake.

Hard protection changes the benefit–harm tradeoff

The main comparison plots mean TTFT improvement against the largest observed matched-request slowdown. Farther right means lower mean TTFT; lower means

less harm to the worst request. Lines connect measured parameter settings only. The plot contains no fitted or extrapolated policy curves.

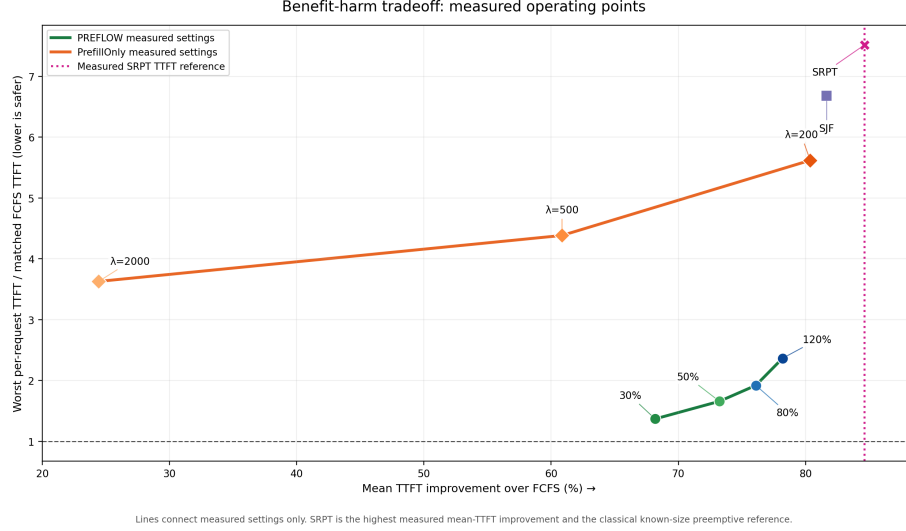


Figure 2: Measured TTFT benefit versus worst-request harm

The measured operating points are:

Policy	Mean TTFT improvement over FCFS	Worst matched-request slowdown
SRPT	84.6%	7.52×
SJF	81.7%	6.68×
PrefillOnly $\lambda = 200$	80.4%	5.62×
PrefillOnly $\lambda = 500$	60.9%	4.38×
PrefillOnly $\lambda = 2000$	24.4%	3.63×
EDF 120%	5.8%	1.60×
PREFLOW 30%	68.2%	1.37×
PREFLOW 50%	73.2%	1.66×
PREFLOW 80%	76.1%	1.92×
PREFLOW 120%	78.2%	2.37×

On this workload, PREFLOW reaches nearly the same mean-latency regime as aggressive aging while moving the worst-request slowdown into a very different range. PrefillOnly $\lambda = 200$ improves mean TTFT by 80.4% with a 5.62× worst slowdown; PREFLOW 120% improves it by 78.2%, 2.2 percentage points lower, with a 2.37× worst slowdown. PREFLOW 30% is better on both measured axes

than the most protective tested PrefillOnly setting: 68.2% versus 24.4% mean improvement and $1.37\times$ versus $3.63\times$ worst slowdown.

SRPT minimizes mean flow time in the classical known-size preemptive single-server model and gives the highest measured mean-TTFT improvement among the policies in this experiment. We therefore use it as the aggressive efficiency reference, not as a formal bound for the real PD implementation. PREFLOW 120% retains about 92% of SRPT’s measured mean-TTFT improvement while cutting worst observed slowdown by more than threefold.

EDF isolates the contribution of the feasibility mechanism. It uses the same frozen deadlines but always follows deadline order, removing PREFLOW’s greedy choice among safe actions. Its 5.8% mean improvement shows that deadlines provide protection, while the work-aware choice among safe actions produces most of the latency benefit.

These points describe the tradeoff measured on this workload and seed. They do not establish a complete Pareto frontier.

A hard modeled constraint produces a different tail

The slowdown tail illustrates why tuning an aging coefficient is not equivalent to enforcing a completion bound. A softer priority can make large slowdowns rarer, but it cannot make the next unsafe overtake illegal.

PREFLOW enforces deadlines in charged, modeled work units inside a run. The plot divides observed wall-clock TTFT under PREFLOW by observed TTFT for the matched request in a separate FCFS execution. The numerator therefore includes cost-model error, asynchronous execution, admission effects, KV transfer, and run-to-run noise; the denominator also differs from the frozen internal baseline B_i .

For that reason, the measured endpoints are $1.369\times$, $1.658\times$, $1.920\times$, and $2.365\times$ for configured modeled bounds of $1.3\times$, $1.5\times$, $1.8\times$, and $2.2\times$. The figure evaluates how the modeled constraint translates to observed wall-clock latency. It is not a direct test of the theorem in its native units.

The horizontal axis is observed TTFT divided by matched FCFS TTFT. The vertical axis is the fraction of all requests slower than that ratio, on a logarithmic scale.

SJF, SRPT, and all three PrefillOnly settings produce tapering tails. Stronger aging shortens the observed tail, but every additional overtake remains a legal action. The endpoints from $3.63\times$ to $7.52\times$ are finite observations from this trace, not policy limits.

PREFLOW has a different shape. Each curve keeps a broad region of useful reordering and then ends near its configured FCFS-relative boundary. The important distinction is not simply that PREFLOW produced a shorter tail in this run. Once the modeled overtaking budget is exhausted, further bypasses

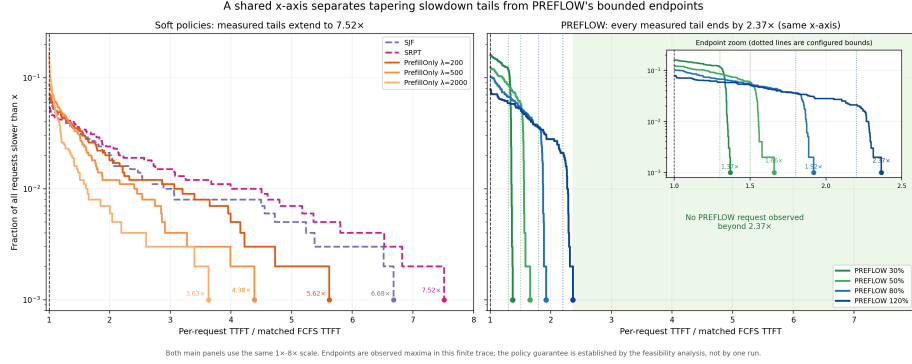


Figure 3: FCFS-relative slowdown tails

become illegal. Changing ρ moves that boundary; it does not change the rule that enforces it.

Queue behavior during bursts

Scheduling cannot create prefill compute. It can change which requests finish first, and that changes a request-count queue because short requests leave the system faster than long ones.

FCFS carries 52.9 waiting requests on average and peaks at 128. EDF behaves similarly, averaging 49.7, because its cumulative FCFS-relative deadlines largely preserve arrival order. SRPT averages 6.5 waiting requests and peaks at 22 by draining short requests first, while PREFLOW 120% averages 9.7 with a $2.37\times$ rather than $7.52\times$ worst matched slowdown. Across the PREFLOW settings, mean waiting occupancy ranges from 15.0 at 30% slack to 9.7 at 120%.

PrefillOnly's queue is sensitive to λ : mean waiting occupancy grows from 9.2 requests at $\lambda = 200$ to 39.6 at $\lambda = 2000$. This is the same tuning tradeoff seen in latency.

Every panel returns to zero near the end of the run. The workload contains temporary bursts, but the system eventually processes all offered work. Lower queue depth here reflects completion order, not extra capacity or admission drops.

How the guarantee works

PREFLOW treats queue slack as an overtaking budget. At a chunk boundary, sort all unfinished released requests by fixed deadline. For the first k requests in that earliest-deadline-first (EDF) order, define

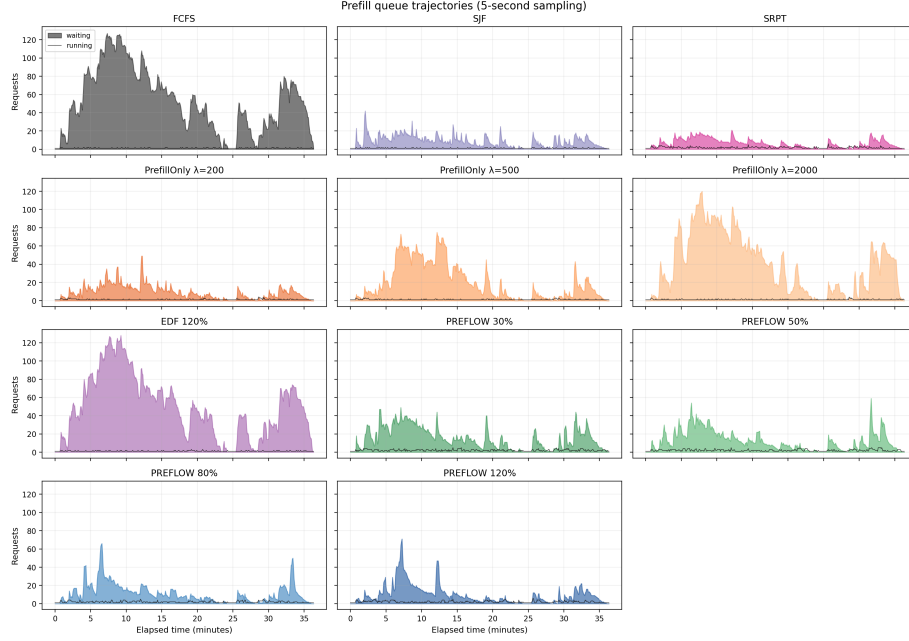


Figure 4: Prefill queue trajectories

$$\sigma_k(t) = D_{(k)} - t - \sum_{j=1}^k r_{(j)}(t).$$

The deadline minus the current clock is the available service; the sum is the work the prefix still needs. A nonnegative σ_k means the prefix remains feasible.

For a candidate at EDF position m with next-chunk cost q , every earlier prefix loses q units of slack. Prefixes containing the candidate lose the same amount of time and remaining work, so their slack is unchanged. The chunk is safe exactly when

$$q \leq \min_{k < m} \sigma_k(t).$$

Prefix slack is the remaining overtaking budget. The earliest-deadline request has no earlier prefix, so it provides a rescue action whenever a preferred overtake is unsafe.

The arrival-state FCFS continuation supplies the initial feasible schedule. Accepted chunks preserve prefix feasibility, and later arrivals cannot change older deadlines. Under conservative cost and admission assumptions, request i therefore completes by $D_i = a_i + \rho B_i$.

From scheduling theory to device time

PREFLOW’s feasibility test needs the service time consumed by a candidate chunk. A model that only ranks requests relative to one another cannot determine whether an overtake fits within the remaining deadline margin.

The vLLM-Ascend implementation profiles the real prefill execution path at startup. Calibration requests run through the deployed model and hardware, and the scheduler measures each chunk directly. For a configured full chunk of size C , it fits a history-dependent cost in milliseconds:

$$T_C(h) = \theta_0 + \theta_1 z + \theta_2 z^2, \quad z = h/H.$$

Here h is the existing token history and H is the history normalization scale used during calibration. The fitted full-chunk curve is constrained to remain non-decreasing across the supported history range.

Short prompts and final partial chunks require separate measurements. Fixed invocation overhead makes their cost non-linear in token count, so PREFLOW profiles a set of sub-chunk sizes instead of scaling a full chunk directly. The profile also measures the additional overhead paid when prefill finalizes.

The scheduler sums these predicted costs to obtain:

- the request’s isolated work;
- its remaining work;
- the predicted cost of its exact next-chunk action; and
- its arrival-state FCFS baseline.

PREFLOW’s logical service clock and its feasibility test therefore use the same device-time units. The profile is local to the deployed model, hardware, parallelism, attention implementation, and chunk size. Once calibration finishes, vLLM-Ascend removes the calibration requests and releases their KV state before serving traffic.

The profiled model is what makes the FCFS-relative completion contract operational on real hardware. The scheduling rule remains separate from the predictor: another additive device-time model can replace this fit without changing the deadline construction or prefix-slack test.

The current implementation fits point estimates from median observations, so the theorem is enforced exactly in charged model units. A deterministic wall-clock interpretation would require conservative charging, such as an upper quantile or a prediction-error envelope.

Making PREFLOW work with a finite KV cache

A request can be urgent according to the compute scheduler and still be unable to run because the resident set occupies the KV cache. Compute-side protection

alone is therefore insufficient. If the engine repeatedly admits younger requests, it can consume the protected request’s deadline margin before that request ever enters the executable set.

PREFLOW uses full-sequence KV reservation to protect admission. Every pending request must retain enough modeled time for two obligations:

1. the resident prefill work blocking its admission must drain; and
2. its own remaining prefill must complete before its fixed deadline.

At logical service time t , let $\mathcal{R}(t)$ be the resident prefill set and order pending requests by deadline. With the conservative full-drain envelope used by the implementation,

$$G(t) = \sum_{j \in \mathcal{R}(t)} r_j(t), \quad t + G(t) + \sum_{j=1}^k r_{(j)}(t) \leq D_{(k)} \quad \text{for every pending prefix } k.$$

Here $G(t)$ upper-bounds the modeled time required for the current resident set to drain and release admission capacity. The prefix inequality requires enough time to remain after that drain for every protected pending prefix to finish by its deadline. A request may be admitted earlier if its reservation fits, but the scheduler does not rely on that outcome for safety.

The scheduler evaluates this envelope as resident work advances and before each admission decision. Younger work cannot consume resident capacity when doing so would make an older fixed deadline infeasible.

This mechanism works with vLLM’s existing waiting/running lifecycle and KV allocator. PREFLOW does not replace the allocator with a separate memory management subsystem. Instead, the safety mechanism covers both decisions that control progress: which resident request receives compute, and which waiting request may become resident.

Without admission-aware protection, a scheduler can satisfy every compute-side priority rule and still violate the intended completion bound because the protected request remained outside the executable set for too long. The admission shield carries the same completion contract from the idealized compute scheduler into a memory-constrained serving engine.

The prefill advantage remains visible end to end

Decode adds another stage after the scheduler-controlled prefill. The same benefit/harm separation remains visible when we measure full end-to-end latency for the 128-token outputs.

PREFLOW spans 66.6% to 76.7% mean E2E improvement, with worst observed E2E slowdown from $1.43\times$ to $2.30\times$. PrefillOnly $\lambda = 200$ reaches 78.8% mean improvement with $5.38\times$ worst slowdown. SRPT reaches 83.4% and $6.95\times$.

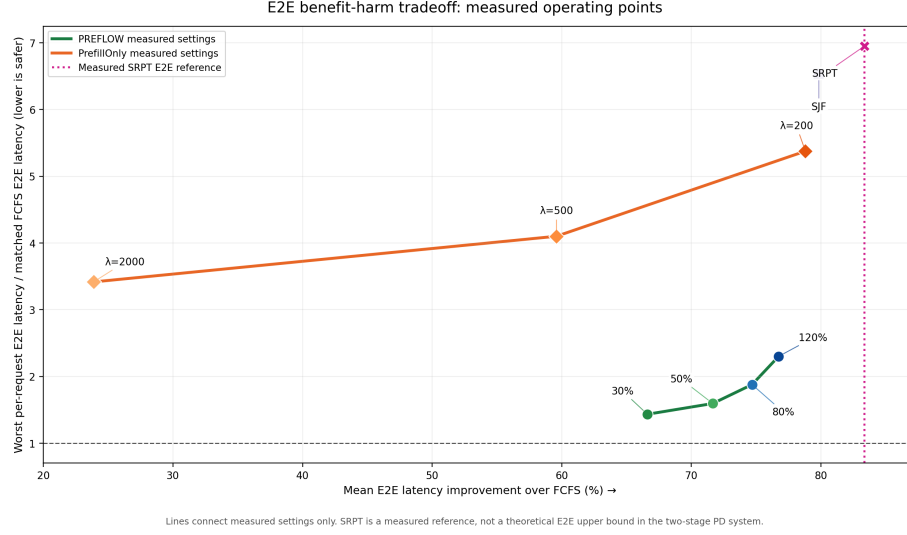


Figure 5: Measured E2E benefit versus worst-request harm

These are measured two-stage results. PREFLOW provides a prefill completion guarantee; it does not provide a full E2E guarantee for downstream decode.

Conclusion

Preemption reduces head-of-line blocking by allowing requests to overtake, but unconstrained reordering can transfer excessive delay to the requests left behind. PREFLOW separates the efficiency objective from the protection mechanism. Work-aware scheduling determines which action is desirable, while FCFS-relative feasibility determines whether that action is safe. The scheduler therefore optimizes within a safety envelope that preserves every fixed FCFS-relative entitlement already in the system.